

Laboratorium nr. 11, laboratorium nr. 12, laboratorium nr. 13	
Imię i nazwisko: Miłosz Dębowski	Kierunek: Informatyka Techniczna
Numer albumu: 415045	Gr. Lab.: 8

Laboratorium nr. 11

Cel ćwiczenia:

Zapoznanie się z podstawowymi mechanizmami przesyłania komunikatów, składnią oraz procesem kompilacji programów przy użyciu MPI.

Przebieg ćwiczenia:

Po przygotowaniu środowiska pracy, odpowiedniej struktury katalogowej oraz zainstalowaniu pakietu mpicc zgodnie z instrukcjami prowadzącego, przystąpiono do implementacji programu MPI_Simple.c.

W pliku Makefile zdefiniowano liczbę procesów, ustawiając ją na 6.

```
#zmiana liczby procesów
run: MPI_simple
    $(MPI_run) -np 6 ./MPI_simple
```

Utworzono tablicę hostname, która służy do przesyłania adresu nadawcy do kolejnych procesów, oraz tablicę receive_hostname, przeznaczoną do odbioru tego adresu przez procesy. Do pobrania adresu nadawcy wykorzystano funkcję gethostname().

```
char hostname[100];
    gethostname(hostname, 100);
```

Do przesłania identyfikatora procesu (ID) oraz nazwy hosta (hostname) wykorzystano funkcję MPI_Send. Wysłane dane obejmowały m.in. rozmiar oraz typ zmiennej, numer docelowego procesu, oznaczenie wiadomości (tag) oraz komunikator COMM_WORLD.

```
MPI_Send( &rank, 1, MPI_INT, dest, tag, MPI_COMM_WORLD );
    MPI_Send( &hostname, 100, MPI_CHAR, dest, tag+1, MPI_COMM_WORLD);
```

Do odbioru identyfikatora procesu (ID) oraz nazwy hosta (hostname) użyto funkcji MPI_Recv. Odebrane dane obejmowały m.in. rozmiar (tablica o wymiarze 100 dla nazwy hosta i zmienna o wymiarze 1 dla ID), typ zmiennej, numer docelowego procesu, tag wiadomości (umożliwiający rozróżnienie procesów w przypadku przesyłania więcej niż jednej wiadomości), komunikator oraz status operacji.

```
MPI_Recv( &ranksent, 1, MPI_INT, i, 0, MPI_COMM_WORLD, &status );
    MPI_Recv( &receive_hostname, 100, MPI_CHAR, i, 1, MPI_COMM_WORLD, &status );
```

Na koniec wypisano ID procesu oraz hostname nadawcy.

Całość kodu prezentuje się następująco:

```
#include <stdlib.h>
#include<stdio.h>
#include<math.h>
#include<unistd.h>
#include "mpi.h"
int main( int argc, char** argv ){
    int rank, ranksent, size, source, dest, tag, i;
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    if(size>1){
        if( rank != 0 ){ dest=0; tag=0;
            char hostname[100];
            gethostname(hostname, 100);
            MPI_Send( &rank, 1, MPI_INT, dest, tag, MPI_COMM_WORLD );
            MPI_Send( &hostname, 100, MPI_CHAR, dest, tag+1, MPI_COMM_WORLD);
        } else {
            char receive_hostname[100];
            for( i=1; i<size; i++ ) {
                MPI_Recv( &ranksent, 1, MPI_INT, i, 0, MPI_COMM_WORLD, &status );
                MPI_Recv( &receive_hostname, 100, MPI_CHAR, i, 1, MPI_COMM_WORLD, &status );
                printf("Dane od procesu o randze (status.MPI_SOURCE ->) %d: %d (i=%d)
otrzymany hostname: %s\n", status.MPI_SOURCE, ranksent, i, receive_hostname );
            }
        }
    }
    else{
        printf("Pojedynczy proces o randze: %d (brak komunikatów)\n", rank);
    }
    MPI_Finalize();
    return(0);
}
```

Wynik:

```
neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_11/MPI_simple$ make run
/usr/bin/mpiexec -oversubscribe -np 6 ./MPI_simple
Dane od procesu o randze (status.MPI_SOURCE ->) 1: 1 (i=1) otrzymany hostname: DESKTOP-CVRD880
Dane od procesu o randze (status.MPI_SOURCE ->) 2: 2 (i=2) otrzymany hostname: DESKTOP-CVRD880
Dane od procesu o randze (status.MPI_SOURCE ->) 3: 3 (i=3) otrzymany hostname: DESKTOP-CVRD880
Dane od procesu o randze (status.MPI_SOURCE ->) 4: 4 (i=4) otrzymany hostname: DESKTOP-CVRD880
Dane od procesu o randze (status.MPI_SOURCE ->) 5: 5 (i=5) otrzymany hostname: DESKTOP-CVRD880
```

Następnie utworzono katalog o nazwie sztafeta i utworzono plik sztafeta.c, gdzie skopiowano początkowo kod z pliku MPI_simple.c i dostosowano do polecenia:

```
#include <stdlib.h>
#include <stdio.h>
#include "mpi.h"
int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int send_value = 10;
    int receive_value;
    if (size > 1) {
        if (rank == 0) {
            MPI_Send(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
            int next= rank+1;
            printf("Proces %d wysłał liczbę %d, do procesu %d\n", rank, send_value,
next);
        }
        else if (rank < size - 1) {
            int next= rank+1;
            int prev= rank-1;
            MPI_Recv(&receive_value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
&status);
            printf("Proces %d odebrał liczbę %d, od procesu %d\n", rank,
receive_value, prev);
            printf("Proces %d wysłał liczbę %d, do procesu %d\n",rank, send_value,
next);
            receive_value++;
            MPI_Send(&receive_value, 1, MPI_INT, (rank + 1) % size, 0,
MPI_COMM_WORLD);
        }
        else {
            int prev = rank-1;
            MPI_Recv(&receive_value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
&status);
            printf("Proces %d odebrał liczbę %d, od procesu %d\n", rank,
receive_value, prev);
        }
    } else {
        printf("Pojedynczy wątek o ranku: %d (brak komunikatów)\n", rank);
    }
    MPI_Finalize();
    return 0;
}
```

Początkowo program zaczyna się od inicjowania MPI oraz pobrania liczby procesów za pomocą `MPI_Comm_Size` oraz własnego identyfikatora (rangi) dla poszczególnego procesu.

```
int rank, size;
MPI_Status status;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

W celu identyfikacji procesów wykorzystano zmienną `rank`, odpowiadającą za rangę każdego procesu. Dostępne procesy podzielono na trzy grupy: proces początkowy, procesy środkowe oraz proces końcowy.

Proces początkowy (o numerze 0) jest odpowiedzialny wyłącznie za przesłanie zmiennej `send_value`, której przypisano wartość 10, do procesu numer 1 za pomocą funkcji `MPI_Send`.

Procesy środkowe obejmują wszystkie wątki poza początkowym i końcowym. Każdy z nich odbiera wartość zmiennej `receive_value` od poprzedniego procesu za pomocą `MPI_Recv`, zwiększa jej wartość o 1, a następnie przesyła ją do kolejnego procesu za pomocą `MPI_Send`.

Proces końcowy odbiera wartość `receive_value` za pomocą `MPI_Recv` i kończy cały przepływ danych, kończąc tym samym "sztafetę".

```
int send_value = 10;
int receive_value;
if (size > 1) {
    if (rank == 0) {
        MPI_Send(&send_value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        int next= rank+1;
        printf("Proces %d wysłał liczbę %d, do procesu %d\n", rank, send_value,
next);
    }
    else if (rank < size - 1) {
        int next= rank+1;
        int prev= rank-1;
        MPI_Recv(&receive_value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
&status);
        printf("Proces %d odebrał liczbę %d, od procesu %d\n", rank,
receive_value, prev);
        printf("Proces %d wysłał liczbę %d, do procesu %d\n",rank, send_value,
next);
        receive_value++;
        MPI_Send(&receive_value, 1, MPI_INT, (rank + 1) % size, 0,
MPI_COMM_WORLD);
    }
    else {
        int prev = rank-1;
```

```

        MPI_Recv(&receive_value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
&status);
        printf("Proces %d odebrał liczbę %d, od procesu %d\n", rank,
receive_value, prev);
    }
} else {
    printf("Pojedynczy wątek o ranku: %d (brak komunikatów)\n", rank);
}

```

Wynik programu sztafeta:

```

neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_11/sztafeta$ make
mpicc -c -g -DDEBUG sztafeta.c
mpicc -g -DDEBUG sztafeta.o -o sztafeta -lm
mpiexec -oversubscribe -np 6 ./sztafeta
Proces 0 wysłał liczbę 10, do procesu 1
Proces 1 odebrał liczbę 10, od procesu 0
Proces 1 wysłał liczbę 10, do procesu 2
Proces 2 odebrał liczbę 10, od procesu 1
Proces 2 wysłał liczbę 10, do procesu 3
Proces 3 odebrał liczbę 10, od procesu 2
Proces 3 wysłał liczbę 10, do procesu 4
Proces 4 odebrał liczbę 10, od procesu 3
Proces 4 wysłał liczbę 10, do procesu 5
Proces 5 odebrał liczbę 10, od procesu 4

```

Następnie na podstawie kodu z wcześniejszego etapu (punkt nr. 5), napisano program równoległy w języku C.

W programie:

Dokonano przesłania zaprojektowanej struktury danych między procesami, wykorzystując spakowany typ danych MPI.

```

MPI_Pack_size(10, MPI_CHAR, MPI_COMM_WORLD, &size_struct);
packet_size = size_struct;
MPI_Pack_size(1, MPI_DOUBLE, MPI_COMM_WORLD, &size_struct);
packet_size += size_struct;
MPI_Pack_size(1, MPI_INT, MPI_COMM_WORLD, &size_struct);
packet_size += size_struct;

```

Każdy proces uczestniczący w przesyłaniu:

Wypisywał zawartość otrzymanej struktury.

Dokonywał modyfikacji zawartości struktury (np. zmieniał wiek, wynik lub dodawał znak do tablicy name).

Wypisywał zmodyfikowaną zawartość struktury przed jej przesłaniem do kolejnego procesu.

```
if(size>1){
    if(rank == 0){
        tag = 0; dest = 1;
        struct My_struct value = {"Paweł", 37, 0};
        MPI_Pack(&value.name[0], 6, MPI_CHAR, bufor, packet_size, &position,
MPI_COMM_WORLD);
        MPI_Pack(&value.age, 1, MPI_INT, bufor, packet_size, &position,
MPI_COMM_WORLD);
        MPI_Pack(&value.points, 1, MPI_INT, bufor, packet_size, &position,
MPI_COMM_WORLD);
        MPI_Send (&rank, 1, MPI_INT, dest, tag, MPI_COMM_WORLD);
        MPI_Send (bufor, position, MPI_PACKED, dest, tag+1, MPI_COMM_WORLD);
    }
    else{
        int prev = rank - 1;
        int next = rank + 1;
        MPI_Recv (&ranksent, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, &status);
        MPI_Recv (bufor, 32, MPI_PACKED, prev, 1, MPI_COMM_WORLD, &status);
        struct My_struct value;
        MPI_Unpack (bufor, 32, &position, &value.name, 6, MPI_CHAR,
MPI_COMM_WORLD);
        MPI_Unpack (bufor, 32, &position, &value.age, 1, MPI_INT,
MPI_COMM_WORLD);
        MPI_Unpack (bufor, 32, &position, &value.points, 1, MPI_INT,
MPI_COMM_WORLD);
        printf("proces %d, odebrał imie: %s, wiek: %d, ilosc punktow: %d od
procesu %d \n", rank, value.name, value.age, value.points, ranksent);
        if(rank < size - 1) {
            position = 0;
            value.points++;
            MPI_Pack(&value.name[0], 6, MPI_CHAR, bufor, packet_size, &position,
MPI_COMM_WORLD);
            MPI_Pack(&value.age, 1, MPI_INT, bufor, packet_size, &position,
MPI_COMM_WORLD);
            MPI_Pack(&value.points, 1, MPI_INT, bufor, packet_size, &position,
MPI_COMM_WORLD);
            MPI_Send(&rank, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
            MPI_Send(bufor, position, MPI_PACKED, next, 1, MPI_COMM_WORLD);
        }
    }
}
```

```

else{
printf("Pojedynczy proces o randze: %d (brak komunikatów)\n", rank);
}
MPI_Finalize();
return(0);
}

```

Uruchomienie programu i wyniki

```

rneox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_11/struktura$ make run
make: Warning: File 'struktura' has modification time 1 s in the future
mpiexec -oversubscribe -np 6 ./struktura
proces 1, odebrał imię: Paweł, wiek: 37, ilosc punktów: 0 od procesu 0
proces 2, odebrał imię: Paweł, wiek: 37, ilosc punktów: 1 od procesu 1
proces 3, odebrał imię: Paweł, wiek: 37, ilosc punktów: 2 od procesu 2
proces 4, odebrał imię: Paweł, wiek: 37, ilosc punktów: 3 od procesu 3
proces 5, odebrał imię: Paweł, wiek: 37, ilosc punktów: 4 od procesu 4
make: warning: Clock skew detected. Your build may be incomplete.

```

Wnioski:

Na przykładzie programu sztafeta.c można zaobserwować standardowy model komunikacji typu punkt-punkt, polegający na bezpośredniej wymianie informacji pomiędzy parą konkretnych procesów. Funkcje `MPI_Send` oraz `MPI_Recv` charakteryzują się blokującym sposobem działania — wykonanie programu zostaje wstrzymane do momentu zakończenia operacji wysyłania lub odbioru wiadomości. Dzięki temu synchronizacja oraz sterowanie przebiegiem programu są łatwiejsze w obsłudze.

Alternatywą dla funkcji blokujących są ich nieblokujące odpowiedniki: `MPI_Isend` oraz `MPI_Irecv`. Te funkcje przekazują sterowanie dalszym instrukcjom natychmiast po ich wywołaniu, umożliwiając asynchroniczną komunikację i równoczesne wykonywanie innych zadań.

Możliwe jest także stosowanie kombinacji funkcji blokujących i nieblokujących, w zależności od specyfiki rozwiązywanego problemu. Oba rodzaje funkcji mają swoje zalety i znajdują zastosowanie w różnych scenariuszach programistycznych.

Laboratorium nr. 12

Cel ćwiczenia:

Nauka programowania z wykorzystaniem komunikacji za pomocą MPI.

Przebieg ćwiczenia:

Po utworzeniu struktury katalogów i pobraniu wymaganych plików zgodnie z instrukcjami prowadzącego, przeprowadzono instalację środowiska MPI. Następnie przystąpiono do implementacji programu obliczającego wartość liczby π , bazując na dostarczonym pliku `oblicz_PI.c`, wprowadzając niezbędne modyfikacje dla równoległej wersji opartej na MPI.

Wartość zmiennej `max_liczba_wyrazow`, określającej liczbę wyrazów do przetworzenia, została wczytana i przesłana do procesów za pomocą funkcji `MPI_Bcast()`. Podział wyrazów pomiędzy procesy zrealizowano w sposób blokowy, definiując zakres pracy przy użyciu zmiennych `my_start`, `my_stride` oraz `my_end`.

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
if (rank == 0) {
    printf("Podaj maksymalną liczbę wyrazów do obliczenia przybliżenia
PI\n");
    scanf("%d", &max_liczba_wyrazow);
}
MPI_Bcast(&max_liczba_wyrazow, 1, MPI_INT, 0, MPI_COMM_WORLD);
int my_start = (max_liczba_wyrazow / size) * rank;
int my_end;
if (rank == size - 1) {
    my_end = max_liczba_wyrazow;
} else {
    my_end = (max_liczba_wyrazow / size) * (rank + 1);
}
```

Następnie dokonujemy obliczeń częściowych wartości π w każdym procesie.

```
for (int i = my_start; i < my_end; i++) {
    int j = 1 + 4 * i;
    local_sum_plus += 1.0 / j;
    local_sum_minus += 1.0 / (j + 2.0);
}
```

Kolejnym krokiem jest obliczenie wyników cząstkowych, gdzie każdy proces przekazuje swoje dane do sumy i różnicy `pi_approx`, wykorzystując funkcję `'MPI_Reduce'` lub `'MPI_Allreduce'`. W przypadku `'MPI_Reduce'` wynik jest zapisywany w procesie wskazanym jako docelowy (tutaj `rank == 0`), natomiast przy użyciu `'MPI_Allreduce'` wynik jest dostępny dla wszystkich procesów.


```
    MPI_Reduce(&local_sum_plus, &global_sum_plus, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
    MPI_Reduce(&local_sum_minus, &global_sum_minus, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
```

Wynik kolejno dla 10, 100, 1000, 10000 wyrazów.

```
neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_12/MPI_pi$ make run
mpirun -oversubscribe -np 6 ./MPI_pi
Podaj maksymalną liczbę wyrazów do obliczenia przybliżenia PI
10
PI obliczone:                3.091623806667839
PI z biblioteki matematycznej: 3.141592653589793
```

```
neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_12/MPI_pi$ make run
mpirun -oversubscribe -np 6 ./MPI_pi
Podaj maksymalną liczbę wyrazów do obliczenia przybliżenia PI
100
PI obliczone:                3.136592684838817
PI z biblioteki matematycznej: 3.141592653589793
```

```
neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_12/MPI_pi$ make run
mpirun -oversubscribe -np 6 ./MPI_pi
Podaj maksymalną liczbę wyrazów do obliczenia przybliżenia PI
100
PI obliczone:                3.136592684838817
PI z biblioteki matematycznej: 3.141592653589793
```

```
neox@DESKTOP-CVRD880:/mnt/d/dev/Programowanie równoległe/lab_12/MPI_pi$ make run
mpirun -oversubscribe -np 6 ./MPI_pi
Podaj maksymalną liczbę wyrazów do obliczenia przybliżenia PI
1000
PI obliczone:                3.141092653621039
PI z biblioteki matematycznej: 3.141592653589793
```

Zwiększenie liczby wyrazów w obliczeniach pozwala na uzyskanie większej dokładności wyników, jednak nie powoduje zauważalnych różnic w czasie wykonania.

Wnioski:

MPI_Reduce jest jedną z funkcji komunikacji grupowej, umożliwiającą agregowanie wartości przesyłanych w ramach komunikatora. Pozwala ona na zdefiniowanie operacji redukującej oraz wskazanie procesu, który tę operację wykonuje. W opisywanym przypadku funkcja ta została użyta do obliczenia sumy całki.

MPI_Allreduce różni się od MPI_Reduce tym, że wynik redukcji jest dostępny we wszystkich procesach, a nie tylko w jednym (tzw. root). Podobnie jak MPI_Reduce, MPI_Allreduce wykonuje operację redukcji, ale następnie rozsyła wynik do wszystkich procesów w komunikatorze.

MPI_Bcast jest używana do rozgłaszania danych do wszystkich procesów w celu ułatwienia podziału zadań.

Laboratorium nr. 13

Cel ćwiczenia:

Doskonalenie umiejętności analizy wydajności programów równoległych.

Dane procesora komputera na którym, było wykonywane ćwiczenie:

```
neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_13/mat_vec/mat_vec_scalability$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:    0-11
Vendor ID:              AuthenticAMD
Model name:             AMD Ryzen 5 7600X 6-Core Processor
CPU family:             25
Model:                  97
Thread(s) per core:     2
Core(s) per socket:     6
Socket(s):              1
Stepping:               2
BogoMIPS:               9399.83
```

Przebieg ćwiczenia:

Po przygotowaniu struktury katalogowej i skopiowaniu pliku ze strony prowadzącego, skompilowano program `calka_omp.c` i wywołano program za pomocą polecenia:

`export OMP_NUM_THREADS=il. Wątków && ./calka_omp`

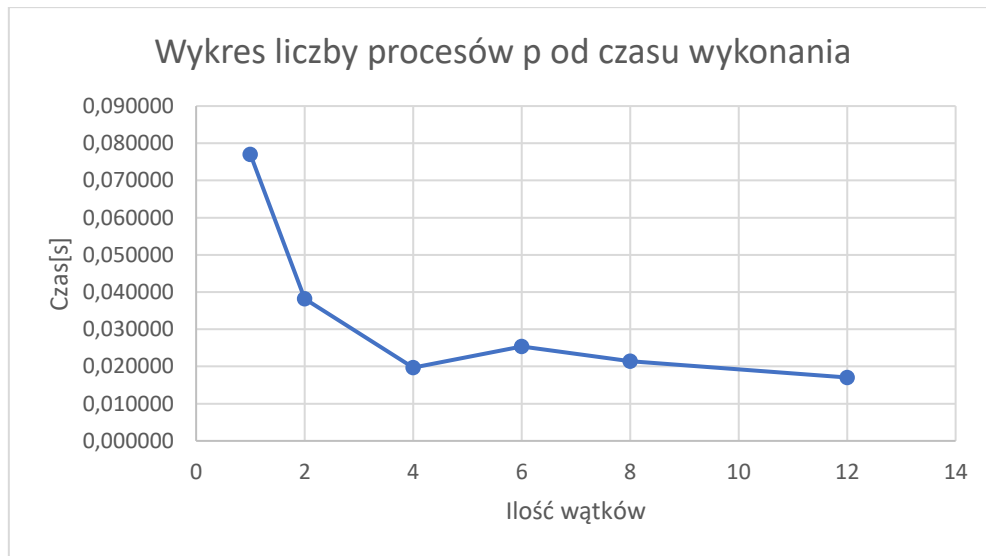
Wywołano program po 3 razy dla każdej ilości wątków (1, 2, 4, 6, 8, 12). Następnie obliczono średnią czasu dla każdego wątku.

Następnie dokonano obliczeń dla efektywności i przyspieszenia względnego:

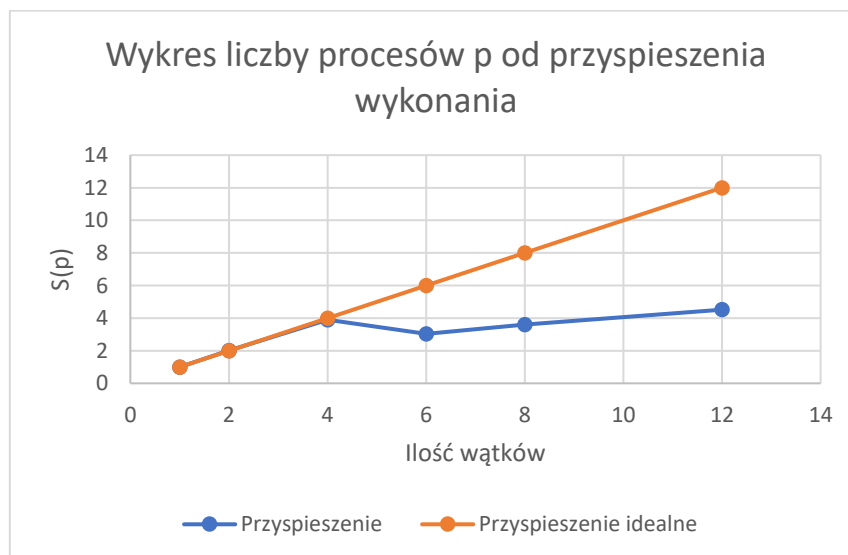
I.w.	Czas wykonania	Przyspieszenie	Efektywność zrównoleglenia	Przyspieszenie idealne	Efektywność idealna
1	0,077019	1	1	1	1
2	0,038239	2,0141567	1,00707835	2	1
4	0,019700	3,909576988	0,977394247	4	1
6	0,025387	3,033783695	0,505630616	6	1
8	0,021406	3,59805036	0,449756295	8	1
12	0,017042	4,51925597	0,376604664	12	1

Następnie zestawiono wykresy zależności od liczby wątków dla:

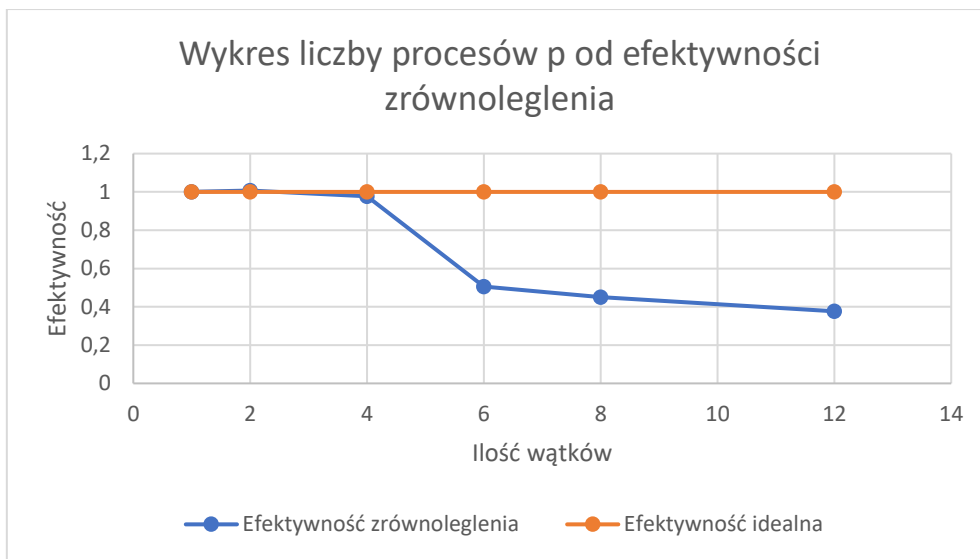
Czasu wykonania:



Przyspieszenia obliczeń:



Efektywności zrównoleglenia:



Następnie utworzono katalog `mat_vec`, gdzie rozpakowano pliki od prowadzącego i uruchomiono kod programu obliczania iloczynu macierz wektor w środowisku MPI dla wymiaru macierzy 12144.

Za pomocą parametru `-np` w pliku `Makefile` zmieniano liczbę wątków oraz za pomocą `--use-hwthread_cpus` włączono hyper threading

```
CCOMP = /usr/bin/mpicc
LOADER = $(CCOMP)
MPIRUN = /usr/bin/mpiexec --use-hwthread_cpus -display-map --bind-to core --map-by socket
OPT = -O3
LIB = -lm
all: mat_vec_row_MPI run

mat_vec_row_MPI: mat_vec_row_MPI.o
    $(LOADER) $(OPT) mat_vec_row_MPI.o -o mat_vec_row_MPI $(LIB)

mat_vec_row_MPI.o: mat_vec_row_MPI.c
    $(CCOMP) -c $(OPT) mat_vec_row_MPI.c $(INC)

run:
    $(MPIRUN) -np 12 ./mat_vec_row_MPI

clean:
    rm -f *.o ./mat_vec_row_MPI
```

Przykładowy wynik pomiaru dla 12 wątków:

```

neox@NEOX:/mnt/d/dev/Programowanie równoległe/lab_13/mat_vec/mat_vec_scalability$ make run
/usr/bin/mpiexec --use-hwthread-cpus -display-map --bind-to core --map-by socket -np 12 ./mat_vec_row_MPI
Data for JOB [64761,1] offset 0 Total slots allocated 12

===== JOB MAP =====
Data for node: NEOX   Num slots: 12   Max slots: 0   Num procs: 12
Process OMPI jobid: [64761,1] App: 0 Process rank: 0 Bound: socket 0[core 0[hwt 0-1]]:[BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 1 Bound: socket 0[core 1[hwt 0-1]]:[../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 2 Bound: socket 0[core 2[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 3 Bound: socket 0[core 3[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 4 Bound: socket 0[core 4[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 5 Bound: socket 0[core 5[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 6 Bound: socket 0[core 0[hwt 0-1]]:[BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 7 Bound: socket 0[core 1[hwt 0-1]]:[../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 8 Bound: socket 0[core 2[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 9 Bound: socket 0[core 3[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 10 Bound: socket 0[core 4[hwt 0-1]]:[../../../../BB/../../../../..]
Process OMPI jobid: [64761,1] App: 0 Process rank: 11 Bound: socket 0[core 5[hwt 0-1]]:[../../../../BB/../../../../..]

=====
Starting MPI matrix-vector product with block row decomposition!
Werja równoległa MPI z dekompozycją wierszową blokową
czas wykonania: 0.015606, Gflop/s: 13.021074, GB/s> 52.089462

```

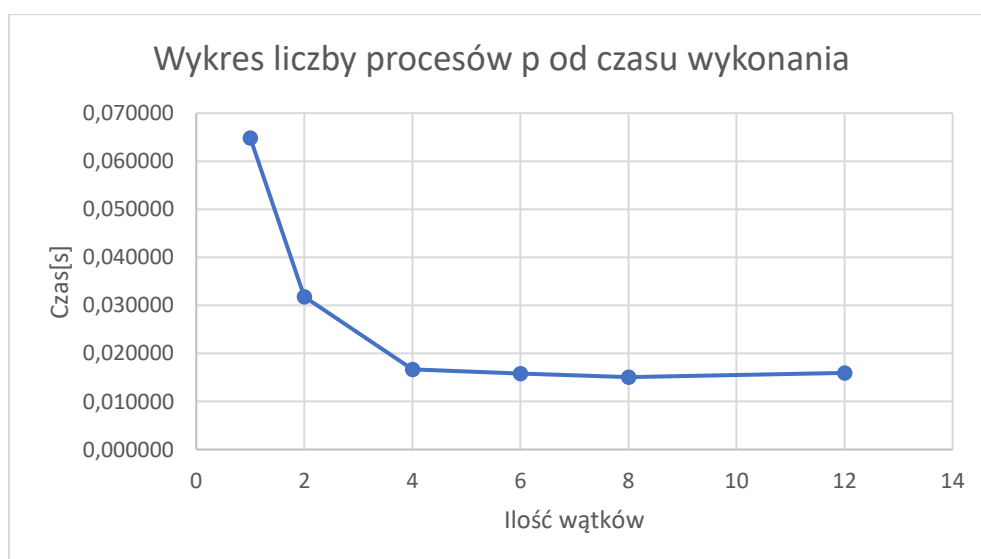
Wywołano program po 3 razy dla każdej ilości wątków (1, 2, 4, 6, 8, 12). Następnie obliczono średnią czasu dla każdego wątku.

Następnie dokonano obliczeń dla efektywności i przyspieszenia względnego:

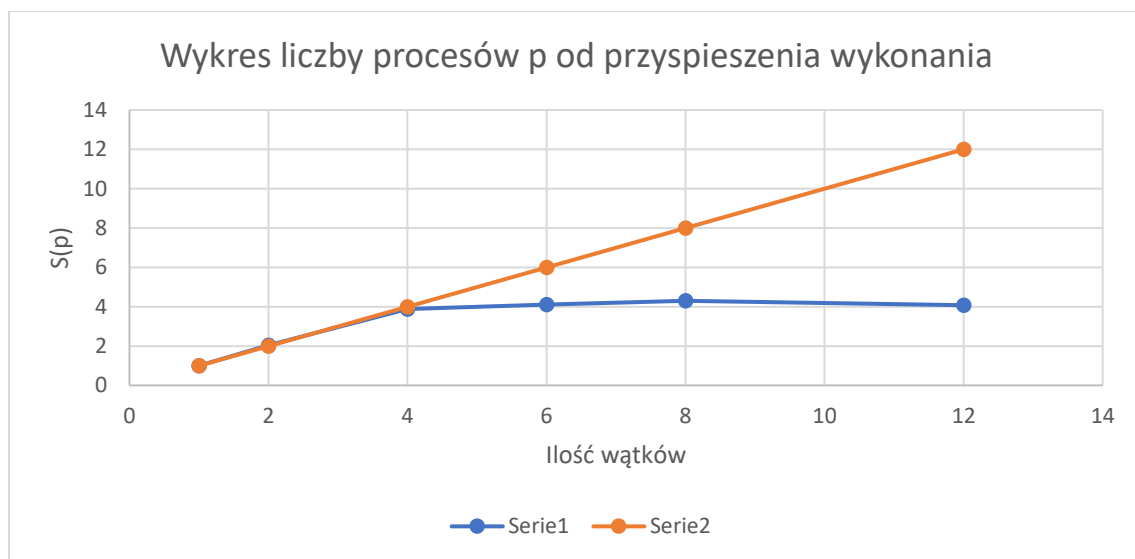
I.l.	Czas wykonania	Przyspieszenie	Efektywność zrównoleglenia	Przyspieszenie idealne	Efektywność idealna
1	0,064820	1	1	1	1
2	0,031817	2,03727567	1,018637835	2	1
4	0,016680	3,886013469	0,971503367	4	1
6	0,015784	4,106603594	0,684433932	6	1
8	0,015073	4,300499801	0,537562475	8	1
12	0,015925	4,070244474	0,33918704	12	1

Następnie zestawiono wykresy zależności od liczby wątków dla:

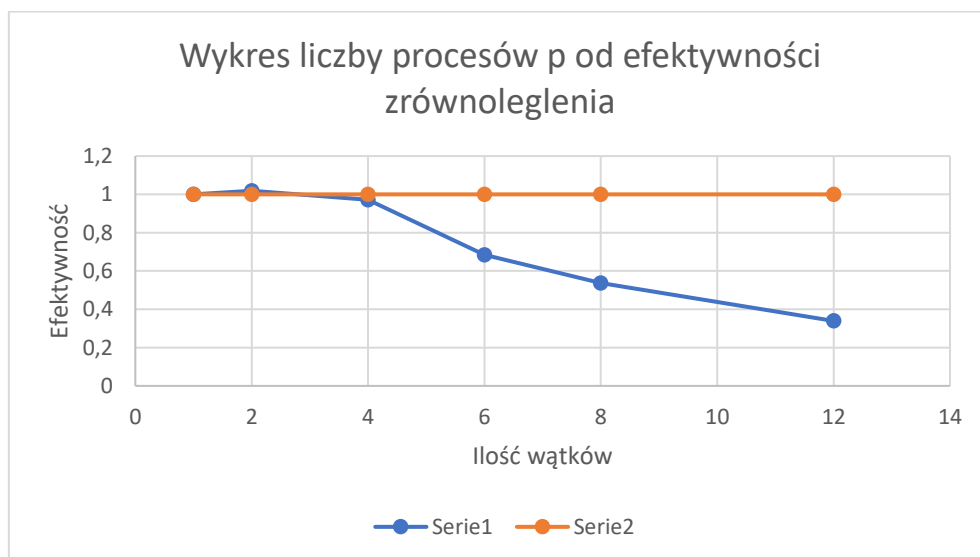
Czasu wykonania:



Przyspieszenia obliczeń:



Efektywności zrównoleglenia:



Całka:

- Dla 1 wątku: Czas wynosi 0,077019, co jest spodziewane, ponieważ wykonanie programu odbywa się sekwencyjnie.
- Dla 2 wątków: Czas spada do 0,038239, co jest prawie idealnym przyspieszeniem (blisko 2 razy szybciej).
- Dla 4 wątków: Czas wynosi 0,019700, co jest bardzo bliskie oczekiwanemu przyspieszeniu (4 razy szybciej).
- Dla 6 wątków: Czas wzrasta do 0,025387, co jest nietypowe, ponieważ oczekiwane jest skrócenie czasu w stosunku do 4 wątków.
- Dla 8 i 12 wątków: Czas wynosi odpowiednio 0,021406 i 0,017042, co sugeruje, że zwiększenie liczby wątków logicznych (przekraczających liczbę rdzeni fizycznych) daje pewne korzyści, ale nieproporcjonalne.

Analiza:

Czas dla 6 wątków:

Wynik dla 6 wątków jest wyraźnie wolniejszy niż dla 4 wątków. To może wynikać z:

- Braku równomiernego podziału obciążenia między rdzenie.
- Narzutów związanych z synchronizacją w programie.
- Zasobów sprzętowych (cache, pamięć) niewystarczających dla 6 wątków jednocześnie.

2. Czas dla 8 i 12 wątków:

Wykorzystanie więcej niż 6 wątków (rdzeni fizycznych) korzysta z hyper-threadingu (SMT). Efektywność SMT zwykle wynosi maksymalnie 30% dodatkowego przyspieszenia. Zatem wynik dla 12 wątków jest zgodny z oczekiwaniami (najkrótszy czas), ale wzrost nie jest proporcjonalny do liczby wątków.

3. Poprawność wyników:

Wyniki wydają się poprawne, choć wskazują na potencjalne problemy z efektywnym zrównolegleniem dla 6 wątków.

Wnioski:

- mogą występować wąskie gardła.
- Oczekuje się liniowego spadku czasu dla liczby wątków $\leq$ liczba rdzeni fizycznych (do 6). Wynik dla 6 wątków sugeruje konieczność analizy synchronizacji lub podziału danych.

Mat_vec:

- Średni czas dla 1 wątku wynosi 0,064820 s.
Jest to punkt odniesienia dla pozostałych wyników. Wykonanie odbywa się w sposób sekwencyjny, co daje najdłuższy czas.
- Średni czas dla 2 wątków wynosi 0,031817 s, co stanowi prawie idealne przyspieszenie (około 2x szybciej w stosunku do 1 wątku).
Wynik sugeruje dobrą efektywność podziału pracy między dwa rdzenie.
- Średni czas dla 4 wątków wynosi 0,016680 s.
Przyspieszenie w stosunku do 2 wątków jest niemal idealne (ponownie około 2x szybciej), co wskazuje na dobrą skalowalność dla liczby wątków mniejszej lub równej liczbie rdzeni fizycznych

- Średni czas dla 6 wątków wynosi 0,015784 s.
Oczekiwano dalszego przyspieszenia, ale różnica względem 4 wątków jest niewielka.

Możliwe przyczyny:

- Brak równomiernego podziału obciążenia: Przy 6 wątkach narzut związany z synchronizacją może znacząco wpłynąć na czas wykonania.
- Wąskie gardła sprzętowe: Cache procesora lub przepustowość pamięci mogą być niewystarczające przy maksymalnym obciążeniu rdzeni fizycznych.

- Średni czas dla 8 wątków wynosi 0,015073 s.

Wprowadzenie wątków logicznych (hyper-threading) daje dodatkowe przyspieszenie, ale nieproporcjonalne w stosunku do liczby wątków.

Hyper-threading zwykle zapewnia przyspieszenie na poziomie do 30%, co jest zgodne z wynikami.

- Średni czas dla 12 wątków wynosi 0,015925 s.

Czas wykonania jest nieco dłuższy niż dla 8 wątków.

Możliwe przyczyny:

- Przepięlenie zasobów sprzętowych: Wykorzystanie większej liczby wątków logicznych może zwiększać narzuty związane z zarządzaniem.
- Konflikty dostępu do pamięci: Przy większej liczbie wątków logicznych wzrasta konkurencja o wspólne zasoby procesora.

Wnioski:

- Problemy z efektywnością dla 6 wątków:
- Warto przeanalizować podział obciążenia i synchronizację w programie, aby zrozumieć, dlaczego wzrost liczby wątków do 6 nie daje wyraźnych korzyści.
- Wpływ hyper-threadingu - Dodatkowe wątki logiczne poprawiają czas wykonania, ale ich efektywność jest ograniczona przez fizyczne zasoby procesora.